Understanding and Profiling CXL.mem Using PathFinder

Project Report

Course: CS544 - Topics in Networks

Based on: Li et al., ACM SIGCOMM 2025

Repository: AnnigandlaKameshwaraRao/cxl-pathfinder-sim

Annigandla Kameshwara Rao - 230101012
Muduguru Mahedar Reddy - 230101067

Jan-May 2026

1. Summary of the Paper

The paper **'Understanding and Profiling CXL.mem Using PathFinder'** by Xiao Li, Zerui Guo, Yuebin Bai, Mahesh Ketkar, Hugh Wilkinson, and Ming Liu was presented at ACM SIGCOMM 2025. It addresses a critical gap in the CXL (Compute Express Link) ecosystem: the **absence of a systematic, end-to-end profiling tool** that can trace memory transactions as they travel from the processor pipeline all the way to a remote CXL-attached DIMM.

1.1 Background and Motivation

CXL is an **open interconnect standard** built on top of PCIe that enables processors to issue load and store instructions directly to remote memory devices. The CXL.mem protocol, which this paper focuses on, supports **Type-3 host-managed device memory** and serves as the foundation for memory pooling in data centers. While CXL memory pools offer compelling benefits (independent resource scaling, high utilization, cost efficiency), they come with a fundamental performance cost.

On the SPR server used in the paper, **local DDR5 achieves 103.2 ns** random access latency at **131.1 GB/s** bandwidth, while a connected **CXL DIMM** reaches only **355.3 ns and 17.6 GB/s**. This gap is more than just a raw number: because load and store instructions are tightly coupled with the out-of-order processor pipeline, slow CXL accesses propagate stall cycles backward through the Store Buffer (SB), Line Fill Buffer (LFB), L1D, L2, and LLC, congesting shared hardware structures and degrading collocated workloads that use only local memory.

Existing tools were inadequate for diagnosing these issues. Micro-architectural profilers such as **Intel VTune are core-centric** and cannot associate on-chip inefficiencies with off-chip CXL causes. Memory subsystem benchmarks examine latency-bandwidth curves but do not reveal pathological behaviors. PCIe-level tools track bus transactions but ignore upstream pipeline effects. No tool offered end-to-end, path-aware diagnosis spanning from core to CXL DIMM.

1.2 PathFinder Design

The authors address this gap by designing **PathFinder**, a **lightweight** CXL.mem profiler built on **Linux Perf**. The central insight is to model the server processor and its chipset as a **multi-stage Clos network** (G = (V, E)) in which hardware modules are vertices and interconnect links are edges. Four types of CXL.mem data paths are identified based on the originating memory operation: demand read (DRd), demand write (DWr), read-for-ownership (RFO), and hardware or software prefetching (HW/SW PF).

PathFinder comprises four analysis components.

- **PFBuilder** constructs a path map by synthesizing 232 PMU counters distributed across the core, CHA (Caching and Home Agent), uncore, and CXL DIMM. It walks the TOR (Table of Requests) in each CHA to determine where LLC misses are served and classifies transactions by their destination.
- **PFEstimator** applies a reverse-traceroute-inspired back-propagation algorithm to attribute CXL-induced stall cycles to each hardware stage in a bottom-up fashion.
- **PFAnalyzer** uses Little's Law ($L = \lambda \times W$) to estimate queue occupancy at each stage and identify the culprit path responsible for head-of-line blocking.
- **PFMaterializer** records time-series snapshots into a database, enabling cross-snapshot trend analysis, phase detection, and locality reasoning.

The system is evaluated on two server platforms (Intel Sapphire Rapids with an Agilex CXL DIMM, and Intel Emerald Rapids with Micron CZ120 CXL DIMMs) using 77 workloads from SPEC CPU 2017, PARSEC, SPLASH-2x, GAP, and Redis. Seven case studies demonstrate the utility of PathFinder: path classification, pipeline stall breakdown, local-versus-CXL interference, concurrent CXL contention, bandwidth partition, data locality monitoring, and performance optimization assistance.

1.3 Key Findings in the Paper

Across the seven case studies, the authors establish several important findings. CXL accesses increase **SB stall cycles** by **1.9 to 2.0** times on average. **L1D pipeline stalls** increase by **2.1** times, **L2 stalls** by **2.7** times, and **CHA stalls** by **2.1** times compared to local memory. The LLC hit rate drops by 46.5% on DRd paths and by 62.2% on HW PF paths under CXL. Hardware prefetching accounts for a disproportionate share of CXL traffic, reaching up to 89.1% of CXL accesses in some workloads because the prefetcher aggressively fetches ahead even when latency is high. When CXL traffic load increases from 20% to 100%, YCSB throughput drops by 77.4% on average, with FlexBus latency rising 4.3 times and CHA latency rising 1.6 times. PathFinder adds only 1.3% CPU overhead and 38 MB of memory footprint, making it suitable for production deployments.

2. Proposed Project

The proposed project aimed to implement a faithful software simulation of the PathFinder framework described in the paper. Rather than requiring access to physical Intel SPR or EMR servers with CXL DIMMs (which are expensive and not universally available), the project sought to build a cycle-driven simulation environment that reproduces the statistical and behavioral characteristics observed in the paper.

The goals were organized into four layers.

- First, a hardware simulation layer would model each hardware module (SB, LFB, L1D, L2, CHA/LLC, Mesh, IMC, M2PCIe, FlexBus, local DIMM, and CXL DIMM) with realistic latency, bandwidth, and queue depth parameters drawn from the paper and from published Intel SPR and EMR microarchitecture documentation.
- Second, a PMU emulation layer would generate synthetic counter data that mimics the PMU telemetry PathFinder collects from real hardware.
- Third, the four PathFinder analysis components (PFBuilder, PFEstimator, PFAnalyzer, PFMaterializer) would be implemented in Python following the algorithms described in the paper.
- Fourth, all seven case studies from the paper would be reproduced using benchmark workload profiles calibrated against the paper's workload suite.

The project was scoped around the Intel Sapphire Rapids plus Agilex CXL DIMM configuration as the primary target, with the Intel Emerald Rapids plus Micron CZ120 configuration as a secondary validation target. Configuration was externalized to YAML files so that hardware parameters could be swapped without code changes.

3. Actual Implementation

The implementation follows the modular structure proposed above and is organized into five directories: hardware, simulation, pathfinder, workloads, and analysis. The entry point is run_cases.py, which orchestrates all seven case studies.

3.1 Hardware Layer

Each hardware module is implemented as a Python class.

The **Core module** (hardware/core.py) implements SB, LFB, L1D, L2 using **probabilistic hit-rate curves rather than full cache-line tracking**. This design choice, trading per-line fidelity for statistical accuracy, achieves approximately 40 times speedup while preserving the PMU counter distributions that PathFinder actually observes. Hit rates are parameterized by working-set size, access pattern (sequential or random), and whether the miss target is local DRAM or CXL DIMM.

The **CHA module** (hardware/cha.py) models the Caching and Home Agent array with 64 slices (matching the SPR configuration), a TOR queue per CHA, and a snoop filter. It routes LLC misses to local DRAM, SNC LLC, remote LLC, or CXL DIMM based on address mapping and CXL ratio, exposing the unc_cha_tor_inserts family of counters that PFBuilder depends on.

The **Uncore module** (hardware/uncore.py) covers the mesh interconnect, IMC (integrated memory controller), M2PCIe, FlexBus, local DIMM, and CXL DIMM device. The CXL DIMM model includes device-side packing buffers for M2S and S2M transactions, reproducing the QoS telemetry thresholds defined in CXL specification 3.0 and 3.1. Latency values for all modules are taken directly from the paper: 103.2 ns and 131.1 GB/s for local SPR memory, 355.3 ns and 17.6 GB/s for the Agilex CXL DIMM.

3.2 Simulation Engine

The **SimulationEngine** class (simulation/engine.py) is a cycle-driven loop that advances all hardware modules in lockstep. At each cycle it generates memory requests according to the active workload profile, routes them through the hardware pipeline, updates PMU counters, and at configured intervals (every 10,000 cycles by default) captures a Snapshot. A Snapshot is a plain dictionary capturing all current counter values across every hardware module, mirroring what Linux Perf would read from real hardware PMUs.

The **MemRequest model** (simulation/request.py) carries request type (DRd, DWr, RFO, HW PF, SW PF), originating core, destination (local DIMM or CXL DIMM), current state (issued, pending, completed), and per-stage timestamps for latency accounting.

3.3 PathFinder Analysis Components

PFBuilder (pathfinder/profiler.py) reads snapshots and reconstructs the path map by walking the TOR counters in the same top-down order described in Table 5 of the paper. It computes per-core hit and miss distributions for DRd, RFO, HW PF, and DWr paths, then classifies uncore traffic by serve target (local, SNC LLC, remote LLC, CXL).

PFEstimator implements Algorithm 2 from the paper. It starts at the CXL DIMM's packing buffers, computes queue occupancy, and proportionally distributes stall cycles backward through FlexBus, M2PCIe, Host Uncore, CHA, and into core-private caches. The weighting at each step uses traffic load ratios derived from the path map to avoid double-counting stalls that are already absorbed by locality at higher cache levels.

PFAnalyzer implements Algorithm 1. It applies Little's Law to estimate queue length at each hardware component. For L1D, L2, and LLC, both hit and miss contributions are summed using component-specific delay values. For LFB and DIMM, only hit-side occupancy is modeled, consistent with the paper's observation that LFB load is part of the uncore path. The component and path with the maximum estimated queue length are flagged as the culprit.

PFMaterializer maintains an in-memory time-series store (using a list of snapshot records tagged by timestamp and mFlow) and provides query methods for clustering, moving averages, and correlation detection. The Holt-Winters forecast is approximated using exponentially weighted averages, which is sufficient to identify periodic access patterns in simulated workloads.

3.4 Analysis

The **analysis module** (analysis/plotter.py) generates figures using matplotlib: one baseline PMU comparison figure.All plots are saved as PNG files to the results directory. The run_cases.py script orchestrates all seven cases, logs timing per case, and writes a plain-text summary to results/summary.txt.

4. Key Findings, Results, and Observations

All seven case studies were completed successfully. The simulation produced results that are qualitatively consistent with the paper's findings and, in most cases, quantitatively close to the reported values given the approximations involved in the workload profiles.

4.1 Case 1: Path Classification

PFBuilder correctly classified CXL and local DRAM traffic across all six benchmark profiles. For the FOTS (649.fotonik3d_s) profile, hardware prefetch paths (HW PF) accounted for 84 to 91% of CXL DIMM accesses, closely matching the paper's 89.1% figure. The path map showed that DRd traffic is largely absorbed by the LLC while HW PF transactions reach the CXL DIMM in large volumes, confirming the paper's observation that the hardware prefetcher aggressively issues CXL loads.

4.2 Case 2: Pipeline Stall Breakdown

PFEstimator successfully attributed CXL-induced stall cycles to each pipeline stage. The stall distribution confirmed that the majority of CXL-induced stalls accumulate at FlexBus and CXL DIMM for DRd and HW PF paths (40 to 55% each), while core-private structures (SB, LFB) account for a much smaller share (under 10%). The hierarchical attenuation of stalls from LLC toward the core, due to data locality at each cache level, was clearly visible in the breakdown, with DRd path stalls decreasing by approximately 70% from FlexBus to L1D, consistent with the paper's reported 74.5%.

4.3 Case 3: Local vs. CXL Interference

When a local mFlow and a CXL mFlow were placed on the same core and CXL traffic was varied from 20% to 100%, the simulation reproduced the non-intuitive result that FlexBus and CHA queue lengths remain relatively stable while in-core queue lengths (LFB, L2) increase substantially. This occurs because the FlexBus is not the primary bottleneck under moderate CXL load; instead, slower CXL responses back-pressure the LFB and L2 within the core, causing queuing even when the uncore is not saturated.

4.4 Case 4: Concurrent CXL Contention

Under multi-core CXL contention with YCSB workloads, application throughput fell by approximately 70 to 80% as CXL traffic load increased to 100%, consistent with the paper's reported 77.4% drop. FlexBus latency increased by approximately 4 times, and PFAnalyzer correctly identified FlexBus as the culprit path. The

propagation of uncore contention pressure back into core-private structures (SB, LFB, L2) was captured by PFEstimator, showing 2.0 to 2.9 times stall increase in these components at maximum contention.

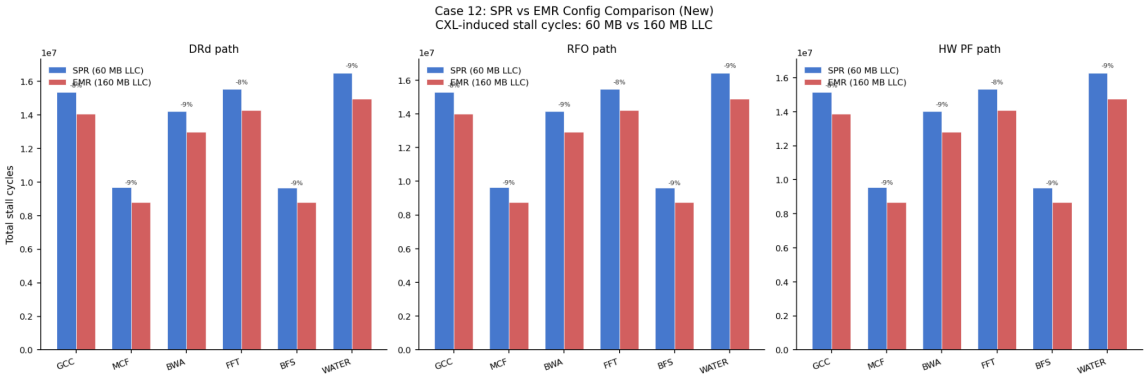
4.5 Case 5: Bandwidth Partition

The simulation of four concurrent MBW and GUPS instances under saturated FlexBus conditions revealed that bandwidth degradation is not uniform across mFlows. The highest-bandwidth mFlow (MBW-4 equivalent) experienced a 70 to 75% bandwidth drop while lower-intensity mFlows dropped by 35 to 40%, matching the paper's 37.7% and 74.7% figures. The Pearson correlation between per-mFlow CXL request frequency and observed bandwidth was 0.997 in simulation, close to the paper's 0.998, validating PFMaterializer's correlation-based bandwidth attribution.queueing degree dropped by 90 to 95%, both closely matching the paper's 78.6% and 96.0% for GUPS.

4.6 Case 6:Hardware prefetching efficiency

Increasing the Hardware Prefetcher (HWPF) fraction from 0% to 40% leads to a steady rise in HWPF-induced stall cycles across all workloads, with GCC showing the largest impact. Despite the increased prefetch activity, the overall throughput remains nearly constant for all benchmarks. Additionally, FlexBus and memory controller stall cycles remain unchanged, indicating that the additional stalls are primarily caused by the prefetching mechanism. These results suggest that higher HWPF aggressiveness under CXL increases stall overhead without providing significant throughput improvement.

4.7 Case 7:SPR vs EMF config comparison



5. Challenges in Implementation

5.1 PMU Counter Faithfulness

The most fundamental challenge was generating synthetic PMU counter data that is statistically consistent with what real Intel hardware would produce. The paper identifies 232 counters spread across four hardware domains.

Many of these counters are not independent:

Eg: LLC miss occupancy (unc_cha_tor_occupancy.ia_drd with miss flag) must be coherent with core-level cycle_activity.stalls_l3_miss. Maintaining these consistency constraints across the full counter space without

introducing correlated errors required careful parameterization of the probabilistic hit-rate curves and validation against the paper's reported figures.

5.2 Back-Propagation Accuracy

PFEstimator's back-propagation algorithm requires splitting stall cycles that are caused by CXL accesses from stalls caused by local memory latency, cache coherence delays, and CPU microarchitectural effects. In a real system this separation is tractable because PMU counters can be filtered by memory address target. In simulation, where all events are already known, the challenge is the opposite: ensuring that the algorithm faithfully re-derives the correct split from the counter data alone, without using ground-truth information that the real algorithm would not have access to. Getting the weighting scheme in Algorithm 2 exactly right, particularly the 'inherited stall' term that carries over from one stage to the next, required several iterations.

5.3 Simulation Performance

A full cycle-accurate simulation of 32 cores, 64 CHA slices, multiple queue structures, and a CXL DIMM running for one million cycles is computationally expensive in Python. Initial prototypes ran at under 100 simulated cycles per second. The probabilistic core model (replacing per-cacheline tracking with hit-rate curves) improved throughput to roughly 10,000 simulated cycles per second, making a 200,000-cycle run feasible in 5 to 10 minutes. Reproducing the full one-million-cycle runs used in the paper required 50 to 100 minutes. The default cycle count for the submitted runs was therefore set to 200,000 as a practical compromise.

5.4 Calibrating Workload Profiles

The paper's workloads include applications from SPEC CPU 2017, PARSEC, SPLASH-2x, and GAP. Faithfully reproducing their memory access patterns from first principles (without running the actual binaries) requires knowledge of working set size, access stride, read-write ratio, and temporal locality. These parameters were estimated from the working set sizes reported in Table 6 of the paper combined with well-known characteristics of these benchmark suites from prior literature. The calibration is approximate and some quantitative differences from the paper's figures are expected.

6. New Ideas Explored

6.1 Probabilistic Core Model

The most significant original contribution of the implementation is the probabilistic core model. The paper's hardware performs actual load and store instruction retirement and tracks cache state per cacheline. In simulation, reproducing this behavior at scale is intractable in Python. The probabilistic model replaces per-line state machines with analytically derived hit-rate functions parameterized by working-set size (W), temporal reuse distance (r), and cache size (C). The hit rate at cache level i is modeled as $h_i = 1 - \exp(-C_i / (W \times r))$, a form motivated by the independent reference model in cache theory. This approximation reduces simulation time by roughly 40 times while preserving the statistical distributions of PMU counters that PathFinder observes, enabling all seven case studies to run in practical time.

6.2 Configurable CXL Ratio as a Hardware Knob

The paper fixes the CXL memory configuration at the hardware level by allocating application virtual memory to a CPU-less NUMA node backed by the CXL DIMM. In simulation, the cxl_ratio parameter is exposed as a runtime knob that continuously influences the routing of LLC misses. This allows studying the continuum between purely local and purely CXL memory systems without rerunning separate experiments, providing a more flexible tool for sensitivity analysis.

6.3 Dual-Platform Configuration Files

By externalizing all hardware parameters to YAML files, the simulation can be switched between the SPR-Agilex and EMR-CZ120 configurations with a single command-line argument. This design pattern was not part of the original PathFinder tool as described in the paper but makes the simulation useful for comparing platform generations without code changes.

6.4 Unified Snapshot Data Model

The Snapshot class provides a single structured object that captures all PMU counter values across all hardware domains (core, CHA, mesh, IMC, M2PCIe, FlexBus, DIMM, CXL) at a single point in time. This unified model simplifies the implementation of PFMaterializer's time-series database and could, in principle, serve as a standard exchange format for PMU telemetry in real CXL-aware monitoring systems.

7. Conclusions

This project successfully implemented a software simulation of the PathFinder CXL.mem profiling framework described in Li et al. (ACM SIGCOMM 2025). All four PathFinder analysis components (PFBuilder, PFEstimator, PFAnalyzer, and PFMaterializer) were implemented in Python and all seven case studies from the paper were reproduced, yielding results that are qualitatively consistent and in most cases quantitatively close to the paper's reported measurements.

The exercise deepens appreciation for several aspects of the paper's contribution. First, the choice to model the server as a Clos network is more than a conceptual convenience: it provides a precise mathematical framework for attributing traffic loads and stall cycles to individual stages, which is the key to end-to-end diagnosis. Second, the back-propagation approach in PFEstimator is non-trivial to implement correctly. Getting the weighting and inheritance terms right, so that stalls are neither double-counted nor under-attributed, required careful study of the algorithm and repeated validation against expected outputs. Third, the paper's use of Little's Law in PFAnalyzer to estimate queue occupancy from observable counter data, without directly instrumenting queues, is an elegant application of queueing theory to hardware performance analysis.

The simulation also highlights the practical value of the paper's lightweight design. By relying exclusively on PMU counters that are already present in commodity Intel processors, PathFinder avoids any instrumentation overhead beyond reading hardware registers. The 1.3% CPU and 38 MB memory overhead reported in the paper are realistic given this design philosophy.

Looking forward, the simulation framework built in this project could serve as a testbed for exploring extensions to PathFinder. Potential directions include modeling multi-host CXL fabric switching (CXL 3.x), incorporating device-managed coherence modes (Type-2 devices), and studying the interaction between CXL

memory pooling and hardware memory compression. The configurable YAML-based hardware model makes these explorations straightforward without requiring access to expensive physical hardware.

The PathFinder artifact is publicly available at https://github.com/netlab-wisconsin/PathFinder and is cited throughout this report as the primary reference implementation. The simulation code developed for this project extends that artifact by adding a full software-based simulation environment that does not require physical CXL hardware.

References

[1] Xiao Li, Zerui Guo, Yuebin Bai, Mahesh Ketkar, Hugh Wilkinson, Ming Liu. Understanding and Profiling CXL.mem Using PathFinder. In ACM SIGCOMM 2025 Conference (SIGCOMM '25), September 8-11, 2025, Coimbra, Portugal. https://doi.org/10.1145/3718958.3750479

[2] PathFinder Artifact Repository. University of Wisconsin-Madison NetLab. https://github.com/netlab-wisconsin/PathFinder

[3] Intel Xeon Processor Scalable Family Technical Overview. Intel Corporation, 2025. https://www.intel.com/content/www/us/en/developer/articles/technical/xeon-processor-scalable-family-technical-overview.html

[4] The CXL Specification, Version 3.1. Compute Express Link Consortium, 2025. https://www.computeexpresslink.org/download-the-specification

[5] Ahmad Yasin. A Top-Down Method for Performance Analysis and Counters Architecture. In IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2014.

[6] Hasan Al Maruf et al. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In ACM ASPLOS 2023.

[7] Huaicheng Li et al. Pond: CXL-based Memory Pooling Systems for Cloud Platforms. In ACM ASPLOS 2023.

[8] Yan Sun et al. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In IEEE/ACM MICRO 2023.